**OS & SP**

**Name :** D. Teja Sadwika **Roll No :** 2520030536

**SKILL – 15**

**1.Skill15.c**

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
   int fd;
   pid_t pid;

   /* Open combined.txt for writing.
      O_TRUNC clears old contents. */
   fd = open(
```

```
    "combined.txt",
    O_WRONLY | O_CREAT | O_TRUNC,
    0644
);

if (fd == -1) {
    perror("Error opening combined.txt");
    return 1;
}

/* Create child process */
pid = fork();

if (pid < 0) {
    perror("fork");
    close(fd);
    return 1;
}
```

```
/* Child process */
if (pid == 0) {
    /* Redirect stdout to combined.txt */
    if (dup2(fd, STDOUT_FILENO) == -1) {
        perror("dup2 stdout");
        _exit(1);
    }

    /* Redirect stderr to the same file */
    if (dup2(fd, STDERR_FILENO) == -1) {
        perror("dup2 stderr");
        _exit(1);
    }

    /* Close original descriptor */
    close(fd);
```

```
/* Write to stdout */
dprintf(
   STDOUT_FILENO,
   "This is standard output.\n"
);

/* Write to stderr */
dprintf(
   STDERR_FILENO,
   "This is standard error.\n"
);

/* Execute a command producing both outputs */
execlp(
   "sh",
   "sh",
   "-c",
   "echo Command output; echo Command error >&2",
```

```
        (char *)NULL
    );

    /* Runs only if execlp fails */
    perror("execlp");
    _exit(1);
}

/* Parent process */
close(fd);

/* Wait for child */
waitpid(pid, NULL, 0);

printf("Q15 completed successfully.\n");
printf("stdout and stderr were merged into combined.txt\n");

return 0;
```

}

## OUTPUT :

```
sadwika@DESKTOP-8CQAO1L:~$ nano skill15.c
sadwika@DESKTOP-8CQAO1L:~$ gcc -Wall -Wextra -g skill15.c -o skill15
sadwika@DESKTOP-8CQAO1L:~$ ./skill15
Q15 completed successfully.
stdout and stderr were merged into combined.txt
sadwika@DESKTOP-8CQAO1L:~$
```